

— AUTONOMOUS QUALITY ENGINEERING · BUILT TO SCALE ACROSS LBG

Meet  **Helios.**

# An autonomous Quality Engineering AI Agent.

*Helios — for the sun and illumination: clarity and visibility in quality automation.*

From a single Jira ticket to **generated test scenarios**, **triggered pipelines** and **trusted, auditable evidence written straight back to the ticket** — with minimal human effort. A proof of concept built to drive the MVP inside the Economic Crime Prevention Platform.

● Google ADK + Gemini on Vertex AI

● GKE

● Harness CI/CD

● Jira / Xray

● GitHub · BigQuery · GCS · Neo4j

SCOPE

PoC → MVP blueprint

REACH

ECP: 4 labs · ~40 teams

ADOPTION

Built once · reused as a template

FOUNDATION FOR

Group-wide rollout

Insight Syndicate · Soumya Samal · Astom Ahashie · Michal Jakub Kolanski

# Testing can't keep pace with how fast ECP changes.

ECP products evolve rapidly across complex data flows, integrations and user journeys. Traditional testing leans heavily on manual effort, deep product knowledge and early, clear requirements — slowing feedback and increasing delivery risk.

6

labs across the platform

~40

teams solving the same  
testing problem  
independently

60%

manual test design today  
before a change can ship  
with confidence

## Where teams lose time today

- ! Heavy reliance on manual test design and deep product knowledge
- ! Slow feedback on whether a change is safe to release
- ! Duplicated effort — every team builds its own approach
- ! Inconsistent quality and uneven test evidence
- ! Coverage depends on requirements being clear and early

Give **Helios** a product change — a Jira story, acceptance criteria, an API or config update — and it decides *what* to test and *why*, runs meaningful automated tests, and produces clear evidence stakeholders can trust. Built once and offered as a standard template: a **single QE can adopt it in one sprint — no extra engineer needed**.



### Generate

Reads acceptance criteria and authors Cucumber/BDD scenarios plus Jira *Test* subtasks — opening a GitHub PR with the new features.



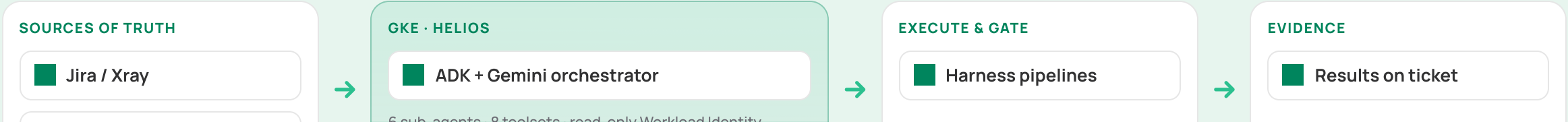
### Execute

Triggers the Harness testing pipelines automatically when a ticket moves to *Testing*, running deterministic + BDD suites against the live pipeline.



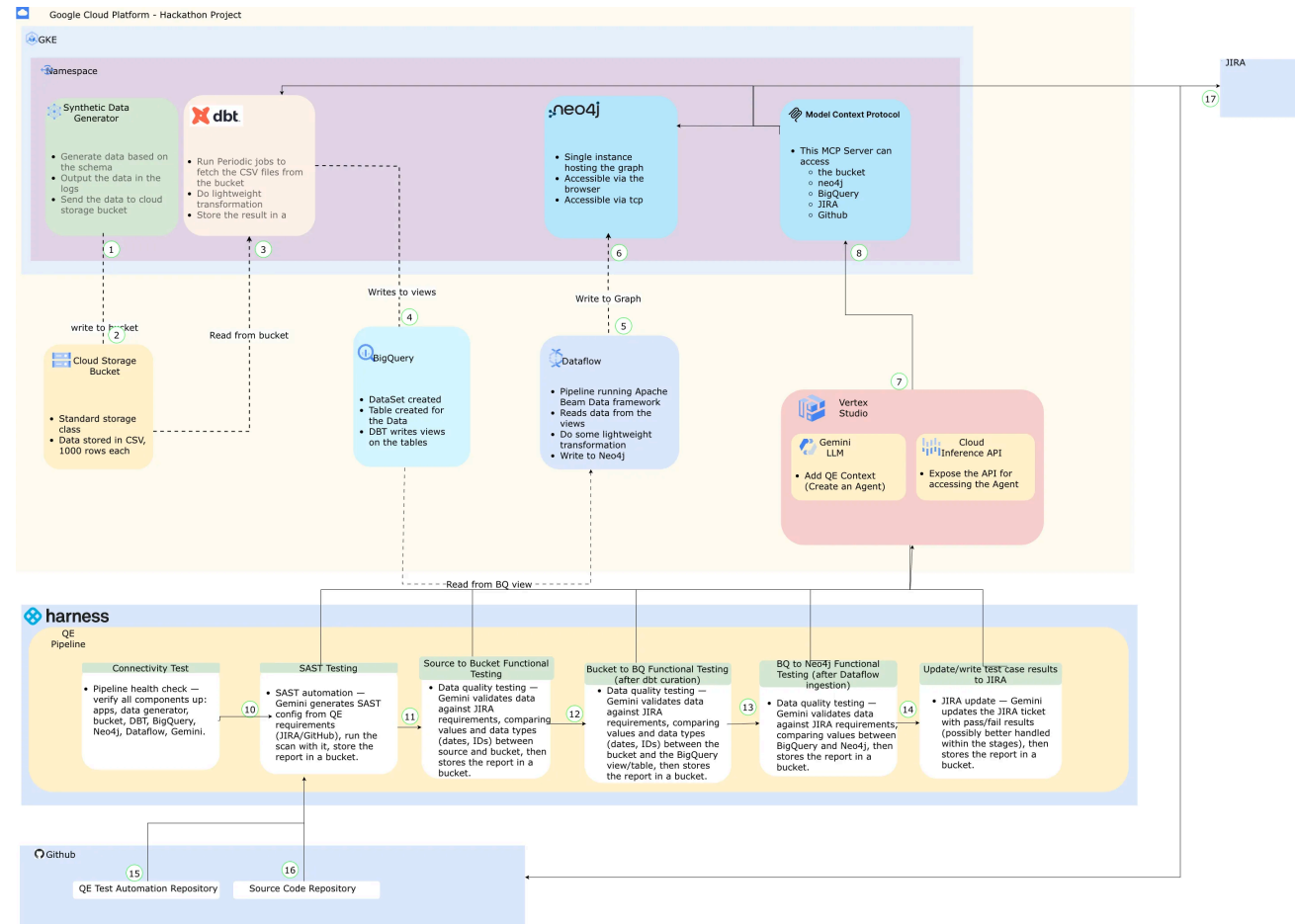
### Evidence

Writes pass/fail results back onto the ticket — updating Xray or regular subtasks — and triages failures into follow-up PRs. Fully auditable.



# One agent, three connected worlds.

Helios runs as a single Deployment in the `qe-hack-syndicate` namespace on GKE, brokering Jira, GitHub and Harness while reading the data pipeline it tests (GCS → dbt/BigQuery → Neo4j) and reasoning with Gemini on Vertex.



# One codebase: an orchestrator, six sub-agents, eight toolsets.

A quick walk through the repo — the ADK + Gemini orchestrator delegates each quality goal to a specialist sub-agent, every one backed by deterministic toolsets.

```
qe-agent/agent

agent/
├─ root_agent.py      # ADK + Gemini orchestrator
├─ config.py          # models · project · toolset wiring
├─ results.py         # PASS / FAIL / ERROR contract
├─ scenarios/         # 6 deterministic QE suites
│   ├── static_analysis.py
│   ├── coding_standards.py
│   ├── integration.py
│   ├── functional.py
│   ├── non_functional.py
│   └─ runner.py
├─ sub_agents/        # specialist Gemini agents
├─ tools/             # gcs·bq·neo4j·dbt·k8s·jira...
└─ server/jira_webhook.py
```

## ORCHESTRATOR → SIX SPECIALIST SUB-AGENTS

static\_analysis

coding\_standards

integration

functional

non\_functional

bdd\_authoring

## EIGHT DETERMINISTIC TOOLSETS (READ-ONLY)

gcs

bigquery

neo4j

k8s

dbt

jira

harness

github

Live walkthrough — root\_agent.py, the sub\_agents/package and the tools/MCP layer. Gemini reasons: the toolsets return a strict PASS / FAIL / ERROR



## GitHub

Code, generated features and agent-authored pull requests.



## Harness

CI/CD pipelines that execute and gate every test suite.



## BigQuery

Raw → enriched data the functional & integration suites assert against.



## Cloud Storage

Raw CSV landing zone for the synthetic data pipeline.



## GKE

# From acceptance criteria to evidence — hands-off.

1

## A ticket is created

Jira emits `issue_created` to the agent. It verifies the token and reads the acceptance criteria.

```
POST /qe/jira/webhook
```

2

## The agent authors the tests

The `bdd_authoring_agent` generates `.feature` files, creates linked Jira *Test* subtasks, and opens a GitHub PR.

3

## The ticket moves to "Testing"

An `issue_updated` status transition fires the Harness `bdd_tests` pipeline via a custom webhook trigger.

4

## Pipelines run the suites

Harness calls the deterministic `/qe/scenario/...` endpoints and the Cucumber pack — the same code path runs in CI, the agent and the CLI.

5

## Evidence is written back

Results are synced onto the ticket's subtasks / Xray execution history. Failing runs are triaged and reconciled into follow-up PRs.

```
POST /qe/jira/{ticket}/sync-results
```

sqlfluff, ruff, bandit, yamllint — zero violations, no hardcoded secrets.



CS1–CS5

## Coding standards

Model naming, documented sources, pinned images, resource limits, FK consistency.



I1–I5

## Integration testing

GCS↔BigQuery parity, no orphan FKs, Neo4j nodes & relationships, watermark integrity.



F1–F6

## Functional testing

Business rules — investment classification, address & phone normalisation, dbt tests.



N1–N2

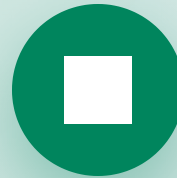
## Non-functional



— LIVE DEMO

## Let's watch it run, end to end.

From a single Jira ticket to trusted evidence — the full loop, live against the running PoC.



Create Jira ticket



Agent authors BDD + PR



Move to "Testing"



Harness runs the suites



Evidence back on the ticket

Demo environment · the live PoC in the `qe-hack-syndicate` namespace on GKE.



## Senior DevOps Engineer

Platform & delivery

- GKE, Workload Identity, RBAC
- Harness pipelines & triggers
- Secrets, networking, observability



## Backend Engineer

Python developer

- ADK agent & toolsets
- Deterministic /qe API
- Jira / GitHub / Harness integration



## Senior Quality Engineer

Test strategy & BDD

- Scenario design & coverage
- Cucumber / Xray integration
- Quality gates & evidence model



on GKE.

| SERVICE                                     | TIER       | ~ / MONTH      |
|---------------------------------------------|------------|----------------|
| GKE node pool (2× e2-standard-2)            | GCP        | £80            |
| Load balancers + networking (2 LBs, egress) | GCP        | £35            |
| Vertex AI – Gemini 2.5 Flash (agent calls)  | GCP        | £20            |
| BigQuery + GCS + Artifact Registry          | GCP        | £10            |
| Persistent disks (Neo4j + pipeline)         | GCP        | £5             |
| Harness                                     | Free trial | £0             |
| Jira / Xray                                 | Free tier  | £0             |
| GitHub                                      | Free       | £0             |
| Neo4j (community) · dbt Core                | Free / OSS | £0             |
| Total PoC run cost                          |            | ≈ £150 / month |

≈ £150 / month

All-in cloud spend for the live PoC

≈ £40–£50 for a typical Hackathon run window. Self-hosted Neo4j on GKE keeps graph costs at zero beyond compute.

Model choice for the PoC

# At production scale, cost per team collapses.

In LBG the platform licences (Harness, Jira/Xray, GitHub Enterprise, GCP org) are already paid — so the marginal cost is a hardened, HA agent footprint shared across many teams. Sizing assumes **~200 pipeline runs/day across ECP** (~4,400 / month).

| COMPONENT                               | NOTE                              | ~ / MONTH      |
|-----------------------------------------|-----------------------------------|----------------|
| GKE (regional, HA agent + supporting)   | Shared cluster, marginal          | £500           |
| Vertex AI — Gemini (authoring + triage) | ~4,400 runs/mo, ~30% LLM-assisted | £200           |
| Networking, LBs & egress                | HA ingress                        | £120           |
| Logging, monitoring & secrets           | Observability                     | £80            |
| Harness · Jira/Xray · GitHub Enterprise | Existing LBG licences             | £0*            |
| Marginal platform cost                  |                                   | ≈ £900 / month |

< £1k

per month, whole platform  
Shared across all adopting teams

~£23

marginal cost per team / month  
At ~40 teams on the shared template

## Model strategy & build-once economics

The PoC runs on **Gemini 2.5 Flash**; for the MVP we step up to **the latest Gemini Pro** model for deeper reasoning at scale. Gemini is invoked only for authoring and failure triage, so each extra team adds just a little usage and compute — onboarding in **one sprint**.

# From Hackathon PoC to reusable platform.

## Q1 · WEEKS 1-4

### Harden the core

Productionise the agent & toolsets, secrets, RBAC, CI/CD on the LBG estate.

## Q1 · WEEKS 5-8

### Prove on a real service

Wire one ECP team's Jira project end-to-end; validate evidence & gating.

## Q1 · WEEKS 9-12

### Template & document

Package the reusable template, onboarding guide and Xray integration.

## Q2 · SCALE

### Roll out & measure

Onboard further teams, track value, and prepare the Group-wide case.

# 1 sprint

for a team to adopt the  
template

# 1-2 Q

to a production-ready MVP

# ~40 teams

addressable on the shared  
pattern

— WHY IT LANDS AT LBG

## Faster feedback, consistent quality, trusted evidence — on tools we already run.

A lean squad delivers a reusable MVP in 1 quarter - allocating just 5–6 hours each per week — running for **under £1.6k a month** across the whole platform — roughly **£40 per team**. Built once, adopted in a sprint, and ready to scale Group-wide.

● Reduce duplicated effort

● Faster, consistent coverage

● Auditable evidence

● Group-wide foundation